

Homework



Homework

- In today's homework, you will create a **ToDo List** app using React.
- You will use a new tool called **Code Sandbox**, which you had wish what Codepen should be. Head to <https://codesandbox.io> to sign in/up.
- Click **Create Sandbox**, and select **React**.
- The interface is divided into 3 sections: *File Explorer*, *Code* and *Browser*.
- **File Explorer**: the default app is scaffolded for you that contains the following:
 - **public** folder with [index.html](#)
 - **src** folder with [App.js](#), [index.js](#) and [styles.css](#)
 - [package.json](#)



Homework

- **Code:** group opened files as tabs along with the active file codes , much like VS Code.
- **Browser:** displays the front end view
- Recap: how react file works
App.js □ index.js □ index.html
- Files:
 - App.js – contains custom scripts of your app rendered as JSX
 - index.js – imports app for publishing to index.html
 - index.html – publish to the front view on the browser



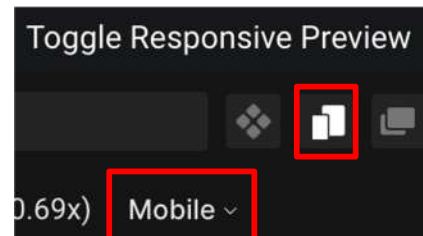
Homework

- What the current app looks like:

Hello CodeSandbox

Start editing to see some magic happen!

- Click the toggle responsive preview to switch between views and to access a specific device view:





Homework

- On the code view, right now you're looking at the app component, which is the only component that's being displayed in our application. On the index.js file, you'll see the app is being imported and rendered within this file.

What does all of this code do?

It simply "renders" or displays the app by injecting it into an index.html file, which is what you see on the preview.

The code also finds and puts the app in the root element (a div with the id of "root") of the index.html file, which is found within the public folder.

```
// src/index.js
import { StrictMode } from "react";
import ReactDOM from "react-dom";

import App from "./App";

const rootElement = document.getElementById("root");
ReactDOM.render(
  <StrictMode>
    <App />
  </StrictMode>,
  rootElement
);
```



Homework

- Now that you have a working React app, let's start building it and changing what you currently see via JSX elements.
- Open App.js, let's begin within the div by removing the h2 element. Within h1, rename the app **Todo List**:

The screenshot shows a code editor with a file named 'App.js'. The code is as follows:

```
1 import './styles.css';
2
3 export default function App() {
4   return (
5     <div className="App">
6       <h1>Todo List</h1>
7     </div>
8   );
9 }
10
```

On the right, a browser preview shows the rendered output: 'Todo List'.

At the bottom of the editor, the status bar indicates: 'Ln 7, Col 11 Spaces: 2 UTF-8 LF JavaScript'.



Homework

- **JSX** as you already aware, looks very similar to HTML, but is in fact JavaScript. We use it to build the structure of our application, just as we would use HTML.
- We can use any standard HTML elements within JSX: divs, heading elements, paragraphs, spans, buttons, etc. It's important to note that there are some minor differences between JSX and HTML.
- The attributes that we use on JSX are slightly different than in normal HTML elements. They are written in the camelCase style, which is the standard way of writing variable or property names in JavaScript.
- For example, to apply a class on a JSX element, use *className* instead of *class* like in normal HTML.

```
<div className="App">  
  <h1>Todo List</h1>  
</div>
```



Homework

- If you use `class` instead of `className` for JSX, you will get a warning in the console that informs you that `class` is an invalid DOM property:

```
3 export default function App() {
4   return (
5     <div class="App">
6       <h1>Todo List</h1>
7     </div>
8   );
9 }
10
```

Todo List

Console Problems React DevTools

Console was cleared

Warning: Invalid DOM property `class`. Did you mean `className`?
at div
at App

Ln 5, Col 14 Spaces: 2 UTF-8 LF JavaScript



Homework

Create a List of Todo Elements

- Since you're creating a todo application, let's create a todo list below the h1 header.
- Begin by making an unordered list with some list items as children elements. Each todo item will be listed within an li element:

```
<div className="App">
  <h1>Todo List</h1>
  <ul>
    <li>Cook Breakfast</li>
    <li>Walk Dog</li>
    <li>Take Shower</li>
  </ul>
</div>
```



Todo List

- Cook Breakfast
- Walk Dog
- Take Shower



Homework

- Okay, that works pretty good right? The todo items displayed as expected.
- However, we can do better as React developers. Let's make a dedicated component that is responsible for displaying the todo items instead.

Create New React Components

- **Components** are the backbone of any React application.
- Use components to separate different parts of the user interface. This makes them reusable wherever you need them across the app. It also organizes your code better, and makes it easier to understand your projects.
- Components fulfill an important concept in programming which is called "separation of concerns." This means it is preferable for each part of your component to have its own clearly defined role and responsibilities, separate from any other component.



Homework

- Just as you have an App component, you can also create a component to be displayed within the App. Since it is a list of todo items, let's call this component **TodoList**:

```
<div className="App">
  <h1>Todo List</h1>
  <TodoList /> { /* component as single tag */ }
</div>
```

Replace the entire unordered list with the component.

- What you have created here is not the component itself but calling or rendering the component, much like how you call/activate a function, *functionName()* in JavaScript.

React Component rules

- Every component must begin with a capital letter. And once a component is declared, it can be written and used very similarly to an HTML element.



Homework

- A component can consist of just one tag or two tags. If it doesn't have anything between the two tags, which are called **children**, it should only have as one tag as the code above displays: `<TodoList />`.
- Additionally, if a component or element consists of just one tag, it must be self-closing. Meaning, it must end in a forward slash (like `<TodoList />` and not `<TodoList>`).
- To create this component for your app, you will write a function component like App, but with the name `TodoList` and without exporting it as it doesn't need to be. Create this function after the App function:

```
}  
  
function TodoList() {  
  
}
```



Homework

- At this moment, your app will produce this error informing nothing was returned from the render (next slide):

```
1 of 2 errors on the page x
Error
ReactDOM.render(): Nothing was
returned from render. This
usually means a return
statement is missing. Or, to
render nothing, return null.

► 18 stack frames were collapsed.

evaluate
/src/index.js:7:9

4 | import App from './App';
5 |
6 | const rootElement =
  document.getElementById("root");
> 7 | ReactDOM.render(
    |           ^
8 |   <StrictMode>
9 |     <App />
10 |   </StrictMode>,
    |

View compiled
► 10 stack frames were collapsed.
```

That means you need to return something, specifically some JSX elements.

Every child component you create must return JSX elements and components (which must also ultimately composed of JSX).

In this case, you want to return the list of todo items. Let's recreate this list that you want to show.



Homework

- Within the TodoList function:

```
function TodoList() {  
  return (  
    <ul>  
      <li>Cook Breakfast</li>  
      <li>Walk Dog</li>  
      <li>Take Shower</li>  
    </ul>  
  );  
}
```



Todo List

- Cook Breakfast
- Walk Dog
- Take Shower

- Notice that the list of todo items re-display again. That means the calling of the component via JSX `<TodoList />` is working as expected!



Homework

- This app currently display a todo list. Let's expand the app a little to display a second todo list. That means you need to:
 1. Call / render this second todo list
 2. Create the list as a functional component
- First, change the name of the current list component name to TodoList1:

```
<div className="App">
  <h1>Todo List</h1>
  <TodoList1 /> {/* component
</div>
```

```
function TodoList1() {
  return (
```

- Next, call the new list component below the first:

```
<h1>Todo List</h1>
<TodoList1 /> {/* component
<TodoList2 />
```



Homework

- Create the second list component below the first:

```
function TodoList2() {  
  return (  
    <ul>  
      <li>Get Lunch</li>  
      <li>Run a Mile</li>  
      <li>Take Bath</li>  
    </ul>  
  );  
}
```



Todo List

- Cook Breakfast
- Walk Dog
- Take Shower
- Get Lunch
- Run a Mile
- Take Bath



Homework

- Let's make the app more dynamic and interesting. The app will display a todo list based on the day of the week.
- To achieve this, you will need to:
 1. Get the current date from your computer
 2. Extract the day from the date
 3. Create an array containing all 7 day's names
 4. Create a condition to test the current day to display the correct todo list
- To start, you will create a 2 day todo list. You will use the current 2 lists that you already created earlier.



Homework

- First, get the date from your computer:

```
export default function App() {  
  
  let d = new Date(); /* Get current date */  
  
  return (  
    <div className="App">
```

- Next, create an array for days:

```
let d = new Date(); /* Get current date */  
  
let day = new Array(7); /* Create array of days */  
day[0] = "Sunday";  
day[1] = "Monday";  
day[2] = "Tuesday";  
day[3] = "Wednesday";  
day[4] = "Thursday";  
day[5] = "Friday";  
day[6] = "Saturday";
```



Homework

- Extract the day from current date as digit and then use that to find matching index number of array:

```
day[6] = "Saturday";  
  
/* Extract day from date as number and  
find matching index from array */  
let n = day[d.getDay()];
```

- Next, create a variable to store the selected todo list during conditional testing:

```
/* Extract day from date as number and  
find matching index from array */  
let n = day[d.getDay()];  
  
/* Create a variable to store selected  
todo list */  
let todo;
```



Homework

- Now let's create a conditional statement to find which todo list to use based on the current day, and set the correct list component calling to the variable:

```
let todo;  
  
/* Test which todo list based on current day  
and set the correct list to the variable */  
if (n === "Monday") {  
  todo = <ToDoList1 />;  
} else if (n === "Tuesday") {  
  todo = <ToDoList2 />;  
}
```



Homework

- The final piece of the puzzle is to replace the previous calling of the 2 components within the return method with the new variable that contains the correct todo list component calling:

```
return (  
  <div className="App">  
    <h1>Todo List</h1>  
    { /* Render the variable that contains  
       the todo component calling */ }  
    {todo}  
  </div>  
);
```



Todo List

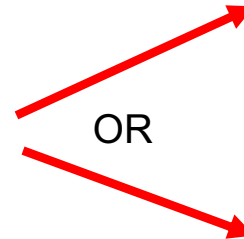
- Cook Breakfast
- Walk Dog
- Take Shower



Homework

- Let's say you want to display the current day on the heading so that you know what day this todo list is for. All you need is add the variable to the heading like so:

```
return (  
  <div className="App">  
    <h1>{n}'s Todo List</h1>  
    { /* Render the variable that contains  
      the todo component calling */ }  
    {todo}  
  </div>  
);
```



Monday's Todo List

- Cook Breakfast
- Walk Dog
- Take Shower

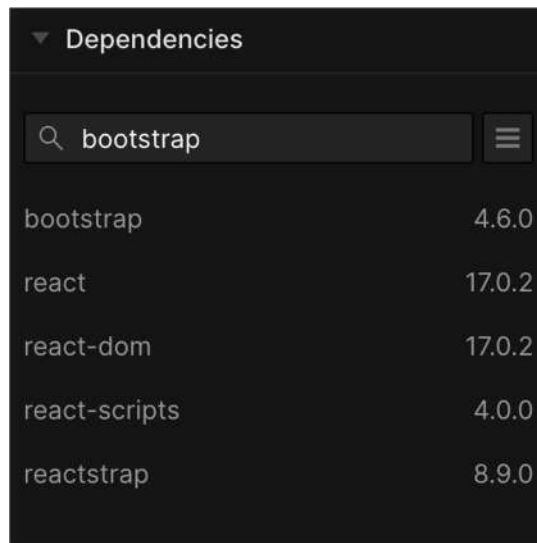
Tuesday's Todo List

- Get Lunch
- Run a Mile
- Take Bath



Homework

- The next phase of the homework for this app is styling it so it looks more presentable. You will use a combination of reactstrap and bootstrap to do this.
- Unlike Codepen, Codesandbox allows us to install dependencies. Head to the dependencies section and enter **reactstrap** and then **bootstrap** to install:



After this, import them into App.js. For reactstrap, import *ListGroup* and *ListGroupItem* to style the list:

```
import "./styles.css";  
import { ListGroup, ListGroupItem } from "reactstrap";  
import "bootstrap/dist/css/bootstrap.min.css";
```

Note: check out a list of reactstrap components and more: <https://react-bootstrap.github.io/components/list-group/>



Homework

- Next, you will change both todo list from `ul` and `li` to `ListGroup` and `ListGroupItem` respectively. This reactstrap should 1) get rid of the list bullets 2) group the list items with some added styles:

```
function TodoList1() {  
  return (  
    <ListGroup>  
      <ListGroupItem tag="a" href="#">  
        Cook Breakfast  
      </ListGroupItem>  
      <ListGroupItem tag="a" href="#">  
        Walk Dog  
      </ListGroupItem>  
      <ListGroupItem tag="a" href="#">  
        Take Shower  
      </ListGroupItem>  
    </ListGroup>  
  );  
}
```

```
function TodoList2() {  
  return (  
    <ListGroup>  
      <ListGroupItem tag="a" href="#">  
        Get Lunch  
      </ListGroupItem>  
      <ListGroupItem tag="a" href="#">  
        Run a Mile  
      </ListGroupItem>  
      <ListGroupItem tag="a" href="#">  
        Take Bath  
      </ListGroupItem>  
    </ListGroup>  
  );  
}
```



Homework

- This is what it should look like in browser preview:

Monday's Todo List

Cook Breakfast

Walk Dog

Take Shower

or

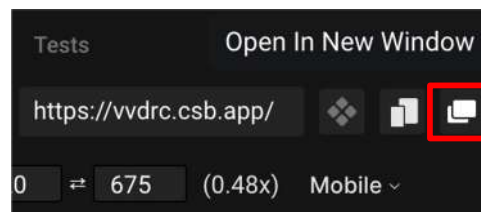
Tuesday's Todo List

Get Lunch

Run a Mile

Take Bath

Note: you can also view this on the actual browser. Click on this button to open on the browser:





Homework

- Next, you will add some bootstrap classes to further style them. Check out this cheat sheet: <https://hackerthemes.com/bootstrap-cheatsheet/>

```
<ListGroupItem
  tag="a"
  href="#"
  className="align-self-center py-2 w-50 bg-info text-white"
>
  Cook Breakfast
</ListGroupItem>
<ListGroupItem
  tag="a"
  href="#"
  className="align-self-center py-2 w-50 bg-info text-white"
>
  Walk Dog
</ListGroupItem>
<ListGroupItem
  tag="a"
  href="#"
  className="align-self-center py-2 w-50 bg-info text-white"
>
  Take Shower
</ListGroupItem>
```

```
<ListGroupItem
  tag="a"
  href="#"
  className="align-self-center py-2 w-50 bg-info text-white"
>
  Get Lunch
</ListGroupItem>
<ListGroupItem
  tag="a"
  href="#"
  className="align-self-center py-2 w-50 bg-info text-white"
>
  Run a Mile
</ListGroupItem>
<ListGroupItem
  tag="a"
  href="#"
  className="align-self-center py-2 w-50 bg-info text-white"
>
  Take Bath
</ListGroupItem>
```



Homework

- On the browser preview:

Monday's Todo List

Cook Breakfast
Walk Dog
Take Shower

or

Tuesday's Todo List

Get Lunch
Run a Mile
Take Bath



Homework

- Next, you will tackle the heading to give it a little more style.
- First, add a comment below return. This is a JSX comment.
- Wrap the week's day with a span element and in it, add a className attribute with some bootstrap classes:

```
return (  
  {  
    /* Output final view */  
  },  
  (  
    <div className="App">  
      <h1>  
        <span className="d-flex justify-content-center text-warning">  
          {n}'s  
        </span>  
        Todo List  
      </h1>  
    </div>  
  )  
)
```



Homework

- On the browser preview:

Monday's Todo List

Cook Breakfast

Walk Dog

Take Shower

or

Tuesday's Todo List

Get Lunch

Run a Mile

Take Bath



Homework

- Let's style the week's day so it stands out more. Add this to the [styles.css](#):

```
h1 span {  
  font-size: 1.3em;  
  display: inline;  
}
```

- On the browser preview:

Monday's
Todo List

- Cook Breakfast
- Walk Dog
- Take Shower

or

Tuesday's
Todo List

- Get Lunch
- Run a Mile
- Take Bath



Homework

- To finish off this homework, you are tasked to add **five** more days of todo lists (Wednesday – Sunday).
- **DUE:**
On or before this Sunday, 10.30PM PT.
- **Submission:**
Post Code Sandbox URL on GAP Week 1 Day 1 Homework



Connect with Us (WEB403/603/803)

Professor

Rich Loke

richloke@westcliff.edu

Teaching
Assistant

Elizabeth Kipp

elizabethkipp@westcliff.edu

Teaching
Assistant

Erin Tustin

erintustin@westcliff.edu

End of Presentation